

ERP 客户差异实现分层规范

1. 文档目的

本文用于约束 ERP 产品化过程中，客户差异应该放在哪里实现。

核心目标是避免所有客户差异都变成：

```
if customer == "xxx"
```

或者全部变成随意 Hook。

客户差异应该按复杂度和风险逐层处理：

配置
↓
规则
↓
策略接口
↓
流程编排
↓
代码级扩展点

越靠前，越稳定、越安全、越适合产品化。

越靠后，能力越强，但风险越高，越需要限制边界。

2. 总体结论

这五层不是并列关系，而是客户差异处理的升级路径。

配置：改参数，不改逻辑
规则：改判断，不写业务代码
策略接口：换算法，但受接口约束
流程编排：改业务步骤，但受状态机约束
代码级扩展点：写客户代码，但必须隔离和受控

一句话判断：

配置解决显示和开关问题。
规则解决条件判断问题。
策略接口解决复杂算法问题。

流程编排解决业务步骤问题。
代码级扩展点解决客户私有实现问题。

项目原则：

能配置就不要规则，能规则就不要策略接口，能策略接口就不要开放代码级扩展点。

3. 第一层：配置

3.1 配置是什么

配置就是：

不写代码，只改结构化参数。

配置适合表达稳定、可枚举、可默认、可开关的差异。

例如：

字段是否显示
字段是否必填
字段默认值
菜单是否启用
页面列展示
导入模板字段
打印模板选择
编号规则
角色权限绑定
流程节点启用 / 停用
是否允许人工改价
是否启用打样流程

3.2 配置适合解决什么问题

适合这类客户需求：

客户 A 不需要“验厂编号”字段。
客户 B 的销售订单列表要显示“客户款号”。
客户 C 不启用打样流程。
客户 D 的报价单使用另一套打印模板。
客户 E 的业务员同时拥有报价和跟单权限。

这些需求本质上是：

显示不同
开关不同
默认值不同
模板不同
权限绑定不同

不应该写客户代码。

3.3 配置怎么实现

推荐两种方式同时存在。

3.3.1 数据库配置

适合运行时可调整的配置。

推荐表：

```
customer_configs  
field_configs  
page_configs  
menu_configs  
workflow_configs  
permission_configs  
numbering_configs  
template_configs
```

示例：

```
{  
  "customer": "yongshen",  
  "module": "sales_order",  
  "fields": {  
    "customer_style_no": {  
      "visible": true,  
      "required": true,  
      "label": "客户款号"  
    },  
    "factory_audit_no": {  
      "visible": false,  
      "required": false  
    }  
  }  
}
```

3.3.2 客户配置包

适合私有化部署和交付固化。

推荐目录：

```
deployments/  
  yongshen/  
    customer-config.yaml  
    field-config.yaml  
    workflow-config.yaml  
    role-config.yaml  
    template-config.yaml
```

示例：

```
customer: yongshen  
  
features:  
  sample_flow: true  
  purchase_flow: true  
  qc_flow: false  
  
sales_order:  
  fields:  
    customer_style_no:  
      visible: true  
      required: true  
      label: 客户款号  
  
    factory_audit_no:  
      visible: false  
      required: false
```

3.4 配置层边界

配置可以做：

```
开 / 关  
显示 / 隐藏  
必填 / 非必填  
默认值  
模板选择  
角色绑定
```

节点启用
参数值

配置不应该做：

复杂判断
复杂计算
跨对象推导
直接修改状态
直接写业务事实

如果配置开始出现复杂条件，就应该升级到规则。

错误示例：

```
if customer_level == "VIP" and amount > 100000 and region == "EU":  
    discount: 0.85
```

这种已经不是配置，而是规则。

4. 第二层：规则

4.1 规则是什么

规则就是：

用结构化条件表达业务判断。

规则比配置更灵活，但仍然不应该写自由业务代码。

规则适合表达：

如果满足某条件，就禁止。
如果满足某条件，就提示。
如果满足某条件，就需要审批。
如果满足某条件，就提高风险等级。
如果满足某条件，就走某个处理分支。

4.2 规则适合解决什么问题

例如：

订单金额超过 10 万，需要老板审批。
客户信用额度不足，禁止提交订单。
交期小于 7 天，需要加急审批。
损耗率超过标准值，需要主管确认。
出货前必须完成质检。
客户等级为 VIP 时允许特殊折扣。
某类产品必须填写安全认证编号。

这些需求不是简单开关，而是条件判断，所以适合规则。

4.3 规则怎么实现

推荐使用：

规则表
决策表
简单 DSL
表达式引擎

规则表结构可以是：

```
rule_id
customer_id
module
scenario
priority
condition
effect
enabled
version
```

示例：

```
{
  "ruleId": "order_submit_credit_check",
  "customer": "yongshen",
  "module": "sales_order",
  "scenario": "before_submit",
  "priority": 100,
  "condition": "order.amount > customer.creditLimit",
  "effect": {
    "type": "reject",
    "message": "订单金额超过客户信用额度，禁止提交"
```

```
}  
}
```

规则执行时，Product Core 提供上下文：

```
{  
  "order": {  
    "amount": 120000,  
    "currency": "CNY",  
    "deliveryDays": 5  
  },  
  "customer": {  
    "level": "VIP",  
    "creditLimit": 100000  
  },  
  "actor": {  
    "roles": ["sales"]  
  }  
}
```

规则引擎返回结果：

```
{  
  "passed": false,  
  "errors": [  
    {  
      "field": "amount",  
      "message": "订单金额超过客户信用额度，禁止提交"  
    }  
  ]  
}
```

4.4 后端接口建议

```
type RuleContext struct {  
    CustomerID string  
    Module      string  
    Scenario    string  
    Facts       map[string]any  
}  
  
type RuleResult struct {  
    Passed    bool  
    Errors    []RuleError  
    Warnings  []RuleWarning  
    Effects   []RuleEffect
```

```

}

type RuleEngine interface {
    Evaluate(ctx context.Context, input RuleContext) (RuleResult, error)
}

```

Product Core 调用：

```

result, err := ruleEngine.Evaluate(ctx, RuleContext{
    CustomerID: order.CustomerID,
    Module:     "sales_order",
    Scenario:   "before_submit",
    Facts: map[string]any{
        "order":    order,
        "customer": customer,
        "actor":    actor,
    },
})

if err != nil {
    return err
}

if !result.Passed {
    return result.Errors
}

```

4.5 规则层边界

规则可以返回：

通过 / 不通过
 错误信息
 警告信息
 是否需要审批
 需要哪个角色审批
 推荐折扣
 风险等级

规则不应该直接做：

修改订单状态
 写库存
 写财务
 删除数据

直接跳流程
直接保存数据库

规则只负责判断，最终动作必须由 Product Core 执行。

5. 第三层：策略接口

5.1 策略接口是什么

策略接口就是：

Product Core 定义稳定接口，客户包可以替换具体实现。

它适合处理配置和规则表达不了的复杂算法。

策略接口本质上已经是代码扩展，但它是强约束代码扩展。

5.2 策略接口适合解决什么问题

例如：

复杂报价算法
复杂物料需求计算
复杂 BOM 展开
复杂审批人选择
复杂生产排程建议
复杂出货拆分逻辑
复杂导入字段映射
复杂外部系统同步转换

这些不是简单条件判断，而是算法、计算、转换，所以适合策略接口。

5.3 策略接口怎么实现

Product Core 定义接口：

```
type PricingPolicy interface {  
    Calculate(ctx context.Context, input PricingInput) (PricingResult, error)  
}
```

定义输入：

```

type PricingInput struct {
    CustomerID string
    ProductID  string
    Quantity   int
    Currency   string
    DeliveryAt time.Time
    Materials  []MaterialCost
    Config     PricingConfig
}

```

定义输出：

```

type PricingResult struct {
    Price          decimal.Decimal
    Currency        string
    TaxRate         decimal.Decimal
    DiscountRate    decimal.Decimal
    Items           []PricingItem
    Explanation     string
}

```

默认实现：

```

type DefaultPricingPolicy struct{}

func (p *DefaultPricingPolicy) Calculate(
    ctx context.Context,
    input PricingInput,
) (PricingResult, error) {
    // 标准报价逻辑
}

```

客户实现：

```

type YongshenPricingPolicy struct{}

func (p *YongshenPricingPolicy) Calculate(
    ctx context.Context,
    input PricingInput,
) (PricingResult, error) {
    // 永绅客户专属报价逻辑
}

```

注册中心：

```

type PolicyRegistry struct {
    pricingPolicies map[string]PricingPolicy
}

func (r *PolicyRegistry) PricingPolicy(customerID string) PricingPolicy {
    if p, ok := r.pricingPolicies[customerID]; ok {
        return p
    }
    return r.pricingPolicies["default"]
}

```

核心调用：

```

policy := policyRegistry.PricingPolicy(order.CustomerID)

priceResult, err := policy.Calculate(ctx, PricingInput{
    CustomerID: order.CustomerID,
    ProductID:  order.ProductID,
    Quantity:   order.Quantity,
    Currency:   order.Currency,
})

```

5.4 策略接口边界

策略接口可以做：

计算
 转换
 匹配
 选择
 返回决策结果

策略接口不应该做：

直接保存订单
 直接修改状态
 直接写库存
 直接写财务
 直接绕过权限

正确模式：

输入上下文 -> 返回结果

错误模式：

输入上下文 -> 直接操作数据库并改变业务状态

6. 第四层：流程编排

6.1 流程编排是什么

流程编排就是：

不同客户的业务步骤可以不同，但所有步骤仍然受状态机约束。

它解决的是：

有的客户流程少一步
有的客户流程多一步
有的客户一个角色身兼多职
有的客户审批更细
有的客户不需要打样
有的客户需要客户确认
有的客户需要内部评审

流程编排不是随便跳状态，而是通过配置表达不同客户的业务路径。

6.2 流程编排适合解决什么问题

标准流程：

报价 -> 订单确认 -> 打样 -> 采购 -> 生产 -> 质检 -> 出货 -> 结案

客户 A：

报价 -> 订单确认 -> 采购 -> 生产 -> 出货 -> 结案

客户 B：

报价 -> 内部评审 -> 客户确认 -> 打样 -> 采购 -> 生产 -> 客检 -> 出货 -> 结案

这些不是字段差异，也不是单条规则，而是流程结构差异，所以适合流程编排。

6.3 流程编排怎么实现

建议拆成四个概念：

State : 状态
Action : 动作
Node : 流程节点
Transition : 状态流转

示例配置：

```
workflow: sales_order
customer: yongshen

states:
  - draft
  - submitted
  - approved
  - production_ready
  - shipped
  - closed

actions:
  submit:
    from: draft
    to: submitted
    permission: order.submit

  approve:
    from: submitted
    to: approved
    permission: order.approve

  skip_sample:
    from: approved
    to: production_ready
    permission: order.sample.skip
    reason_required: true

  ship:
    from: production_ready
    to: shipped
    permission: shipment.confirm

  close:
```

```
from: shipped
to: closed
permission: order.close
```

后端接口：

```
type WorkflowEngine interface {
    GetAvailableActions(ctx context.Context, input WorkflowContext)
    ([]WorkflowAction, error)
    ExecuteAction(ctx context.Context, input ExecuteActionInput) error
}
```

执行动作时：

```
func (s *OrderService) ExecuteAction(ctx context.Context, input
ExecuteActionInput) error {
    order := s.orderRepo.Get(input.OrderID)

    action, err := s.workflowEngine.ResolveAction(ctx, order.State,
input.Action)
    if err != nil {
        return err
    }

    if !s.permission.Can(input.Actor, action.Permission) {
        return ErrForbidden
    }

    if action.ReasonRequired && input.Reason == "" {
        return ErrReasonRequired
    }

    order.State = action.To

    s.audit.Record(...)

    return s.orderRepo.Save(order)
}
```

6.4 流程少一环怎么实现

不能直接删除业务闭环。

比如客户不需要打样，应该配置成：

```
sample:
  enabled: false
  mode: auto_skip
  skip_reason: "该客户不需要打样流程"
```

或者：

```
actions:
  skip_sample:
    from: approved
    to: production_ready
    auto: true
    reason: "客户配置为免打样"
```

核心仍然要记录：

谁跳过
为什么跳过
什么时候跳过
从什么状态到什么状态
是否系统自动跳过

6.5 流程多一环怎么实现

例如增加“内部评审”：

```
states:
  - draft
  - internal_review
  - customer_confirmed
  - approved

actions:
  submit_review:
    from: draft
    to: internal_review
    permission: order.submit_review

  internal_approve:
    from: internal_review
    to: customer_confirmed
    permission: order.internal_approve

  customer_confirm:
    from: customer_confirmed
```

```
to: approved  
permission: order.customer_confirm
```

必须定义：

- 节点名称
- 进入条件
- 完成条件
- 责任角色
- 需要权限
- 是否阻塞后续流程
- 退回动作
- 审计要求

6.6 流程编排边界

流程编排可以做：

- 定义节点
- 定义动作
- 定义状态流转
- 定义权限要求
- 定义责任角色
- 定义跳过原因
- 定义退回路径

流程编排不应该做：

- 绕过状态机
- 跳过审计
- 直接写库存事实
- 直接写财务事实
- 让页面自己决定状态
- 让客户代码任意跳转状态

流程变化必须仍然保证业务闭环。

7. 第五层：代码级扩展点

7.1 代码级扩展点是什么

代码级扩展点就是：

客户包可以接入自定义代码，但这是最后手段。

它适合处理配置、规则、策略接口、流程编排都表达不了的客户私有逻辑。

代码级扩展点风险最大，所以必须最谨慎。

7.2 代码级扩展点适合解决什么问题

例如：

某客户特殊外部系统协议
某客户特殊文件解析
某客户特殊导入清洗
某客户特殊单据生成
某客户特殊历史系统兼容
某客户特殊数据转换
某客户私有算法

7.3 代码级扩展点怎么实现

不要让客户代码散落在核心里。

推荐目录：

```
server/  
  internal/  
    core/  
      order/  
      material/  
      workflow/  
  
    extension/  
      contract/  
      registry/  
      defaultimpl/  
  
    customers/  
      yongshen/  
        policies/  
        importers/  
        exporters/  
        integrations/
```

客户扩展包只能通过注册中心接入：

```
func RegisterYongshenExtensions(registry *ExtensionRegistry) {
    registry.RegisterPricingPolicy("yongshen", &YongshenPricingPolicy{})
    registry.RegisterImporter("yongshen", "sales_order",
        &YongshenOrderImporter{})
    registry.RegisterExternalSync("yongshen", &YongshenExternalSync{})
}
```

核心启动时加载：

```
func BootstrapExtensions(registry *ExtensionRegistry) {
    defaultimpl.Register(registry)
    yongshen.Register(registry)
}
```

核心调用时只认接口，不认客户实现：

```
importer := registry.GetImporter(customerID, "sales_order")
result, err := importer.Parse(ctx, file)
```

7.4 代码级扩展点边界

客户代码不能随便拿数据库连接乱写。

建议只提供受限 Port：

```
type ExtensionDataPort interface {
    GetProduct(ctx context.Context, productID string) (ProductDTO, error)
    GetCustomer(ctx context.Context, customerID string) (CustomerDTO, error)
    FindMaterials(ctx context.Context, query MaterialQuery) ([]MaterialDTO,
        error)
}
```

不要直接给客户扩展代码：

```
*sql.DB
*ent.Client
OrderRepository
InventoryRepository
```

否则客户代码很容易绕过状态机、权限和审计。

代码级扩展点可以做：

特殊解析
特殊转换
特殊对接
特殊模板生成
特殊格式适配
返回处理结果

不允许做：

直接修改核心状态
直接写库存
直接写财务
直接删除核心数据
直接跨租户访问数据
直接绕过权限
直接跳过审计

8. 五层对比表

层级	本质	解决什么	是否写代码	风险	示例
配置	改参数	显示、开关、默认值、模板	不写	最低	字段隐藏、菜单开关、模板选择
规则	改判断	条件判断、校验、审批条件	不写业务代码	低到中	金额超过 10 万需审批
策略接口	换算法	复杂计算、复杂选择	写受控代码	中	报价算法、BOM 计算
流程编排	改步骤	多一步、少一步、角色不同	主要配置，少量引擎代码	中到高	跳过打样、增加内部评审
代码级扩展点	客户自定义代码	特殊协议、特殊解析、特殊集成	写客户代码	最高	私有导入解析、外部系统对接

9. 判断标准

9.1 用配置

当需求是：

要不要显示
要不要启用
是不是必填
默认值是什么

模板选哪个
角色绑定什么权限

就用配置。

示例：

客户 A 不需要“验厂编号”字段。

实现：

```
fields:
  factory_audit_no:
    visible: false
    required: false
```

9.2 用规则

当需求是：

如果满足某条件，就提示 / 禁止 / 需要审批 / 变更等级

就用规则。

示例：

订单金额超过 10 万，需要老板审批。

实现：

```
{
  "condition": "order.amount > 100000",
  "effect": {
    "type": "require_approval",
    "role": "boss"
  }
}
```

9.3 用策略接口

当需求是：

有复杂计算
有复杂匹配
有复杂转换
规则表达太难
需要客户自己的算法

就用策略接口。

示例：

客户 A 的报价需要根据材料成本、损耗率、汇率、历史订单、客户等级综合计算。

实现：

```
type PricingPolicy interface {  
    Calculate(ctx context.Context, input PricingInput) (PricingResult, error)  
}
```

9.4 用流程编排

当需求是：

流程步骤不一样
节点多了
节点少了
审批顺序不同
角色责任不同
状态流转不同

就用流程编排。

示例：

客户 A 不需要打样，客户 B 需要内部评审。

实现：

```
workflow:
  sample:
    enabled: false
    mode: auto_skip

  internal_review:
    enabled: true
    position: before_customer_confirm
```

9.5 用代码级扩展点

当需求是：

配置、规则、策略接口、流程编排都表达不了
而且这个差异确实是客户私有逻辑

才用代码级扩展点。

示例：

客户有一个老系统，只能导出特殊格式 TXT 文件，需要特殊解析。

实现：

```
type Importer interface {
    Parse(ctx context.Context, file File) (ImportResult, error)
}
```

客户包：

```
type YongshenSpecialTxtImporter struct{}

func (i *YongshenSpecialTxtImporter) Parse(
    ctx context.Context,
    file File,
) (ImportResult, error) {
    // 永绅特殊 TXT 解析逻辑
}
```

10. 推荐实现顺序

本项目不要一开始就把所有东西都做成扩展点。

推荐顺序：

第一步：先做 Product Core 标准模型
第二步：给核心字段、菜单、页面、模板加配置能力
第三步：做规则引擎，只处理校验和审批条件
第四步：抽出少量高价值策略接口
第五步：做 Workflow Engine，支持流程节点启停和动作流转
第六步：最后才开放客户代码级扩展点

注意：

这个顺序不是项目 Phase，也不是硬性的开发阶段。
它是客户差异能力的建设优先级。

11. 测试要求

每一层都必须能测试。

11.1 配置测试

验证：

字段显示正确
字段隐藏后不影响保存
导入模板字段正确
打印模板字段正确
菜单权限正确
默认值正确

11.2 规则测试

验证：

规则命中正确
规则优先级正确
错误信息正确
警告信息正确
审批要求正确
规则关闭后不生效

11.3 策略接口测试

验证：

默认策略可用
客户策略可用
客户策略返回结果符合契约
异常输入能处理
客户策略不能破坏状态机

11.4 流程编排测试

验证：

标准流程能闭环
少一步流程能闭环
多一步流程能闭环
退回动作正确
跳过动作有审计
角色合并后责任仍然可追溯

11.5 代码级扩展点测试

验证：

客户代码只能通过受限接口访问数据
客户代码不能直接写核心状态
客户代码不能跨租户访问数据
客户代码异常不会拖垮核心流程
客户代码结果能被审计

12. 反模式清单

以下做法禁止或强烈不推荐：

所有客户差异都写 `if customer == xxx`
所有差异都做 `beforeSave / afterSave` Hook
配置里塞复杂业务表达式
规则直接修改数据库
策略接口直接保存订单
流程编排绕过状态机
代码级扩展点直接拿 `ent.Client`
客户代码直接写库存、财务、业务事实
客户包复制整套页面再改

客户包复制整套接口再改
没有默认实现
没有契约测试
客户包没有版本声明

13. 与客户扩展点的关系

客户扩展点不是单独的一种技术，而是这些层级中的受控入口。

对应关系：

字段扩展点 -> 配置
校验扩展点 -> 规则 / 策略接口
价格扩展点 -> 策略接口
物料需求扩展点 -> 策略接口
流程扩展点 -> 流程编排
模板扩展点 -> 配置 / 策略接口
外部系统扩展点 -> 代码级扩展点

所以，扩展点不应该直接等于 Hook。

更准确的关系是：

扩展点 = Product Core 允许客户差异进入系统的受控位置

而配置、规则、策略接口、流程编排、代码级扩展点，是扩展点的不同实现方式。

14. 项目评估结论

这份分层规范值得单独成文。

原因：

它决定客户差异放在哪里。
它能防止产品核心被客户代码污染。
它能防止所有差异都变成 Hook。
它能指导 Codex 和开发人员判断实现边界。
它能作为后续客户包、验收、测试的依据。

建议将本文作为架构参考文档，和以下文档放在一起：

```
customer-extension-points.md
customer-difference-implementation-layers.md
customer-package-and-deployment.md
workflow-and-state-machine-guideline.md
business-fact-and-audit-guideline.md
```

最终原则：

客户差异不是不能做，而是必须先分类。

分类清楚之后，优先用配置和规则解决；只有复杂算法才进入策略接口；只有流程结构变化才进入流程编排；只有极特殊客户私有能力才开放代码级扩展点。